

启发式搜索与对抗搜索

晏轶超

2026年4月

饮水思源 · 爱国荣校

目 录

1

启发式搜索算法

2

对抗搜索算法

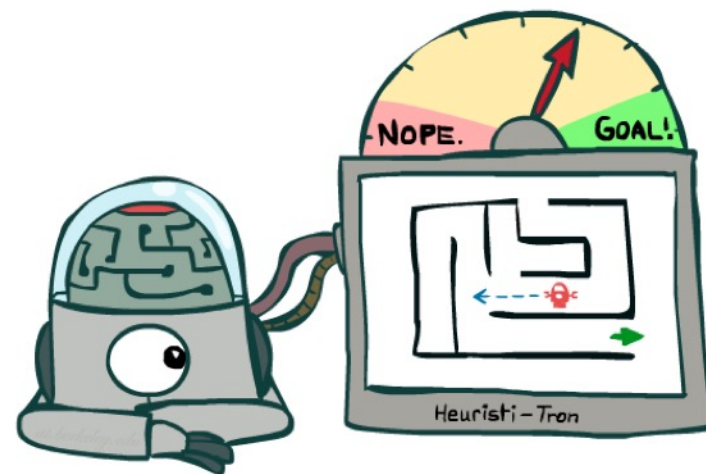
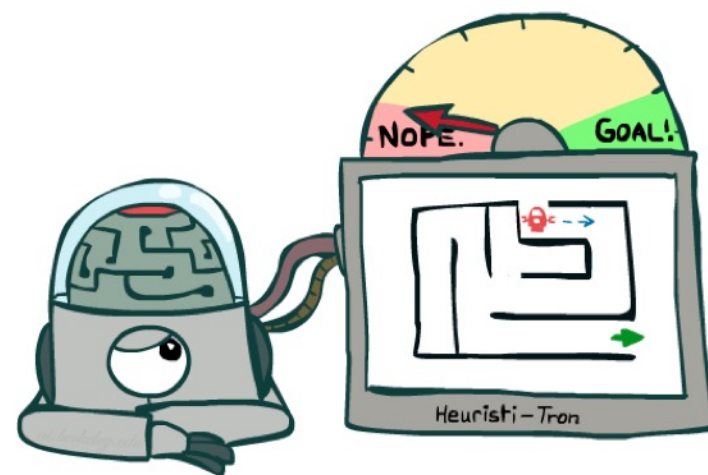
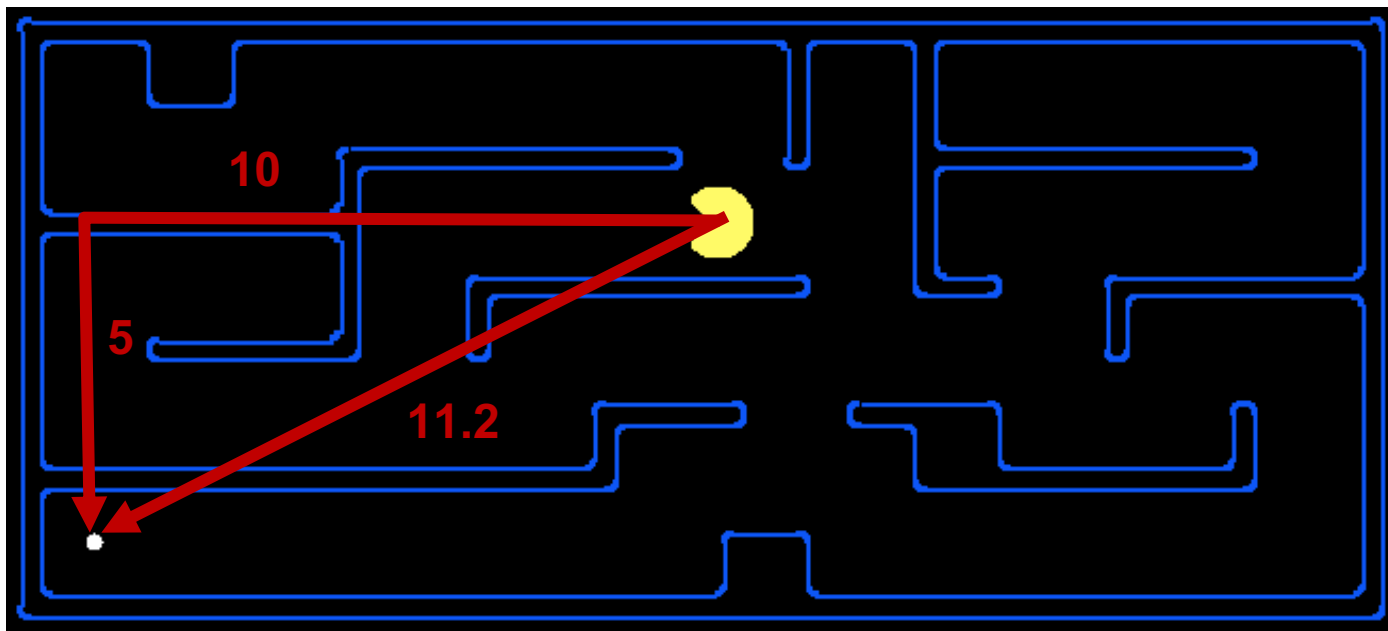


1. 启发式搜索算法 — 启发式函数



▪ 启发式函数 (Heuristics Function)

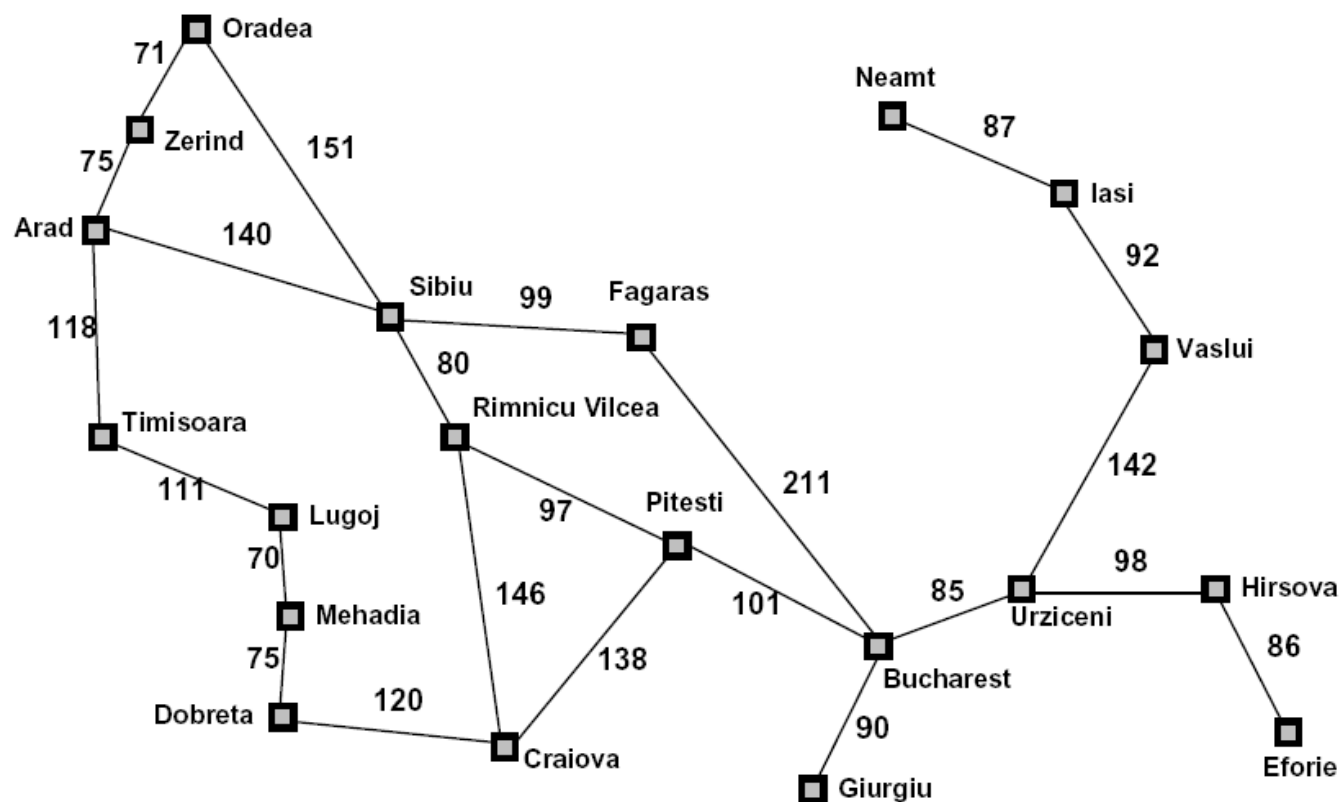
- 预测某一状态距离目标状态远近的函数
- 不同的搜索问题有不同的启发式函数
- 例如：曼哈顿距离、欧几里得距离



1. 启发式搜索算法 — 贪心搜索



1. 启发式搜索算法 — 启发式函数



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

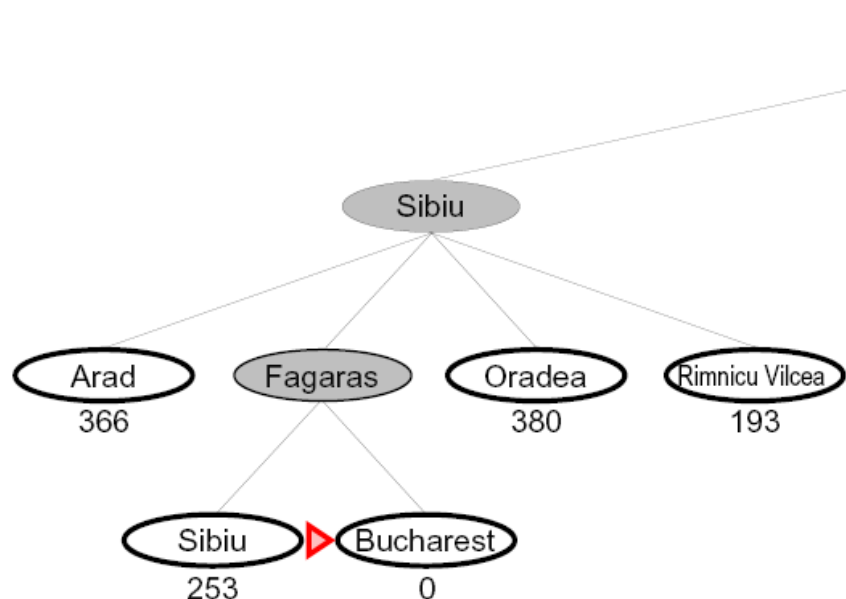
$h(x)$



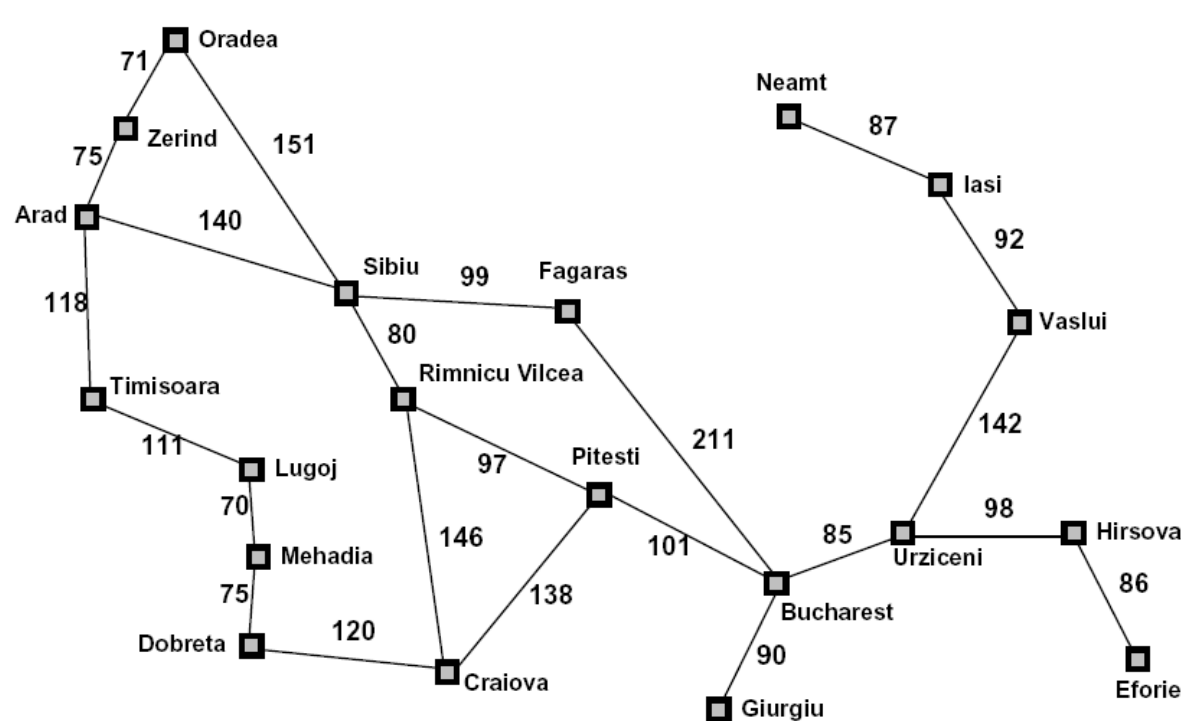


1. 启发式搜索算法 — 贪心搜索

▪ 向距离目标最近的节点搜索



▪ 这一结果存在什么问题？

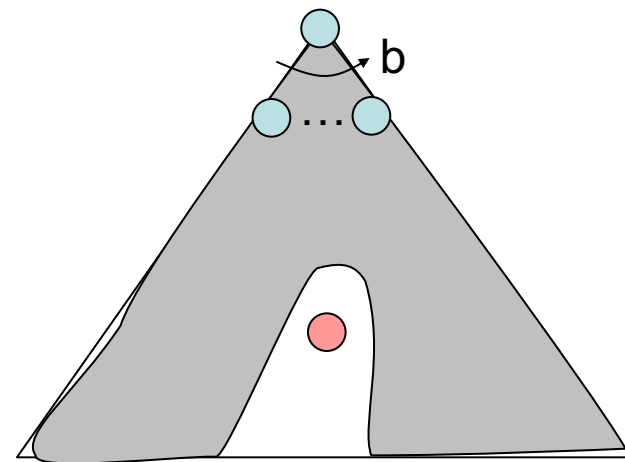
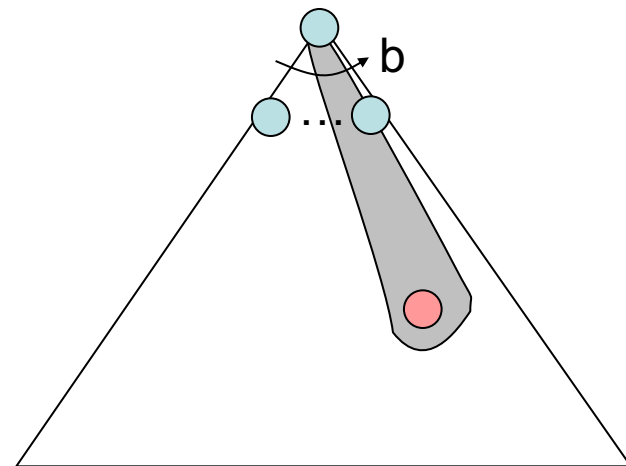
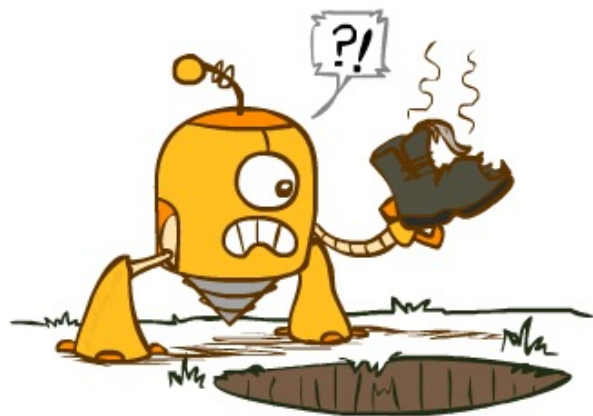


1. 启发式搜索算法 — 贪心搜索



▪ 贪心搜索 (Greedy Search)

- 搜索**被认为是最接近目标状态**的结点
- 启发函数：预测每个状态到最近目标的距离
- 通常情况：最优优先策略会将你带往目标
- 最坏情况：类似于方向错误的深度优先算法



1. 启发式搜索算法 — 贪心搜索



1. 启发式搜索算法 — 贪心搜索



1. 启发式搜索算法 — A*搜索



一致代价搜索



贪心搜索



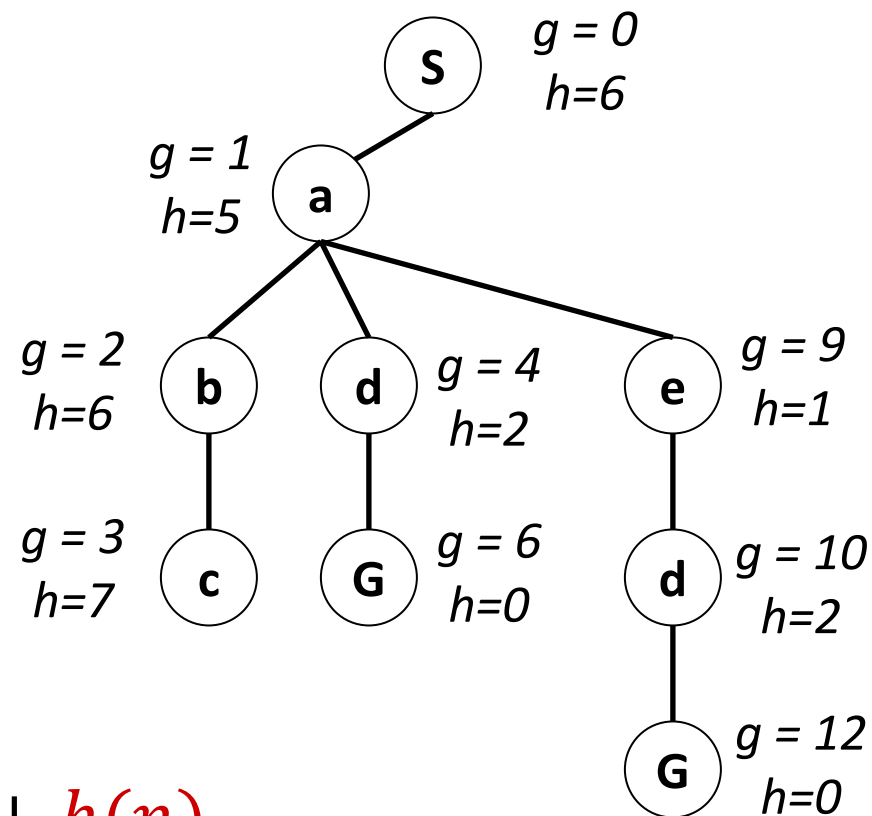
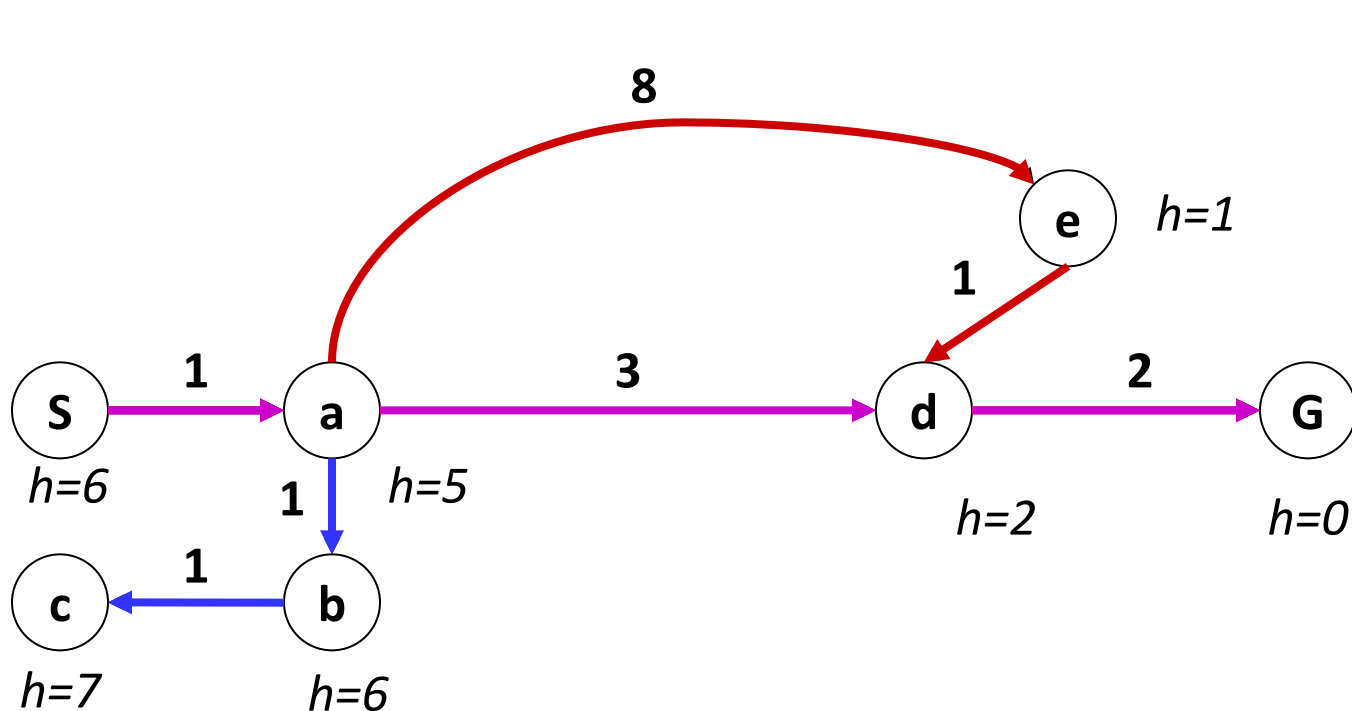
A*搜索

1. 启发式搜索算法 — A*搜索



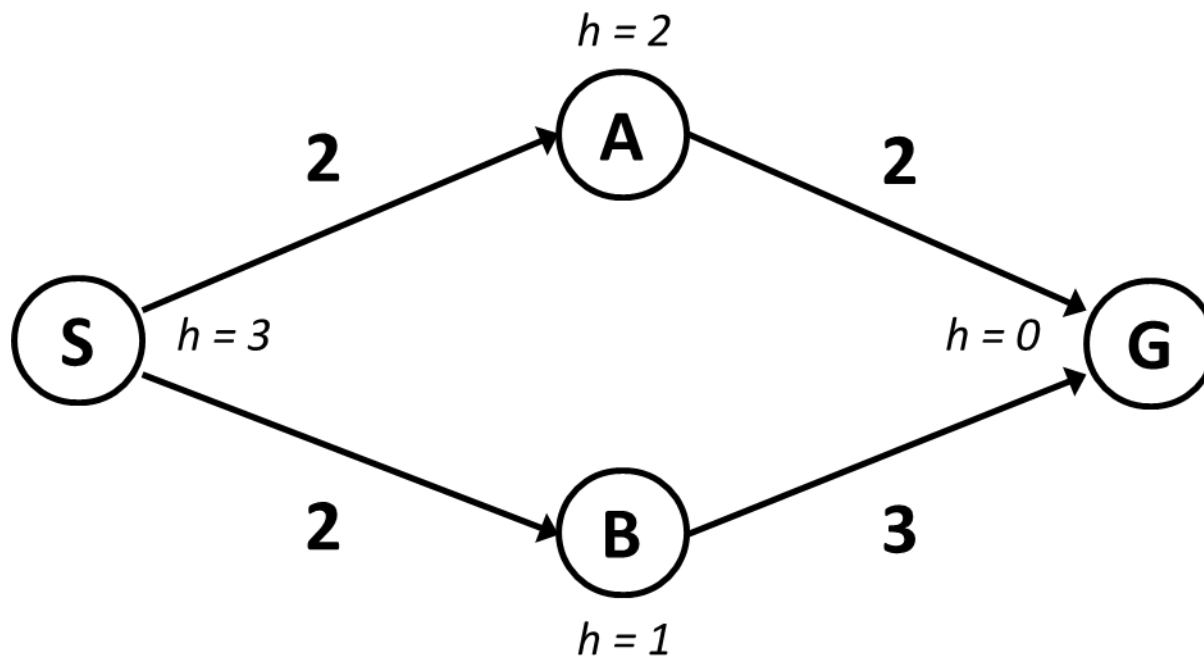
1. 启发式搜索算法 — A*搜索

- **一致代价搜索**以路径代价为依据搜索，即后向代价 $g(n)$
- **贪心搜索**以距离目标远近为依据搜索，即前向代价 $h(n)$



- **A*搜索**计算这两者之和，即： $f(n) = g(n) + h(n)$

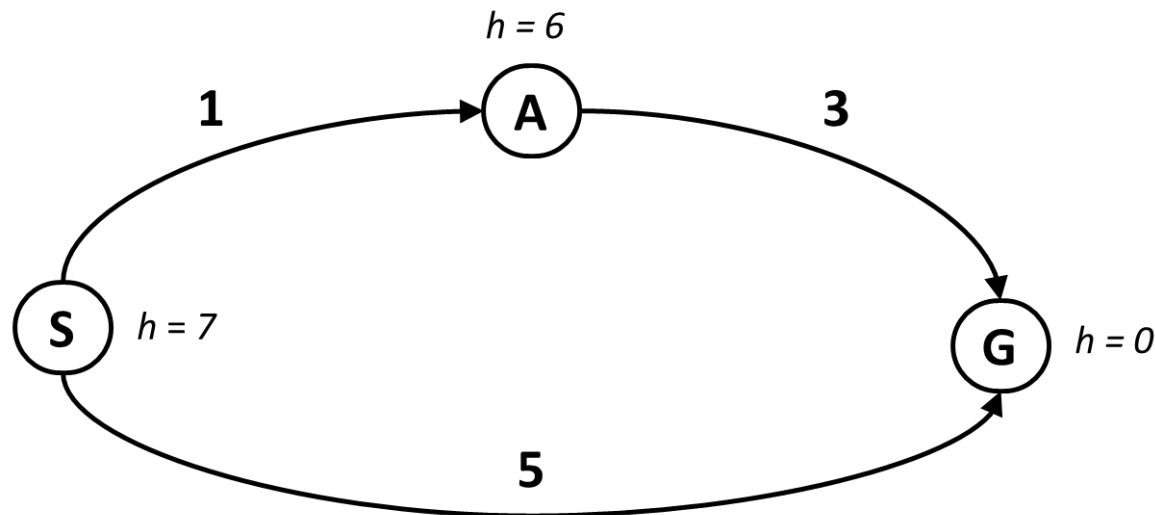
1. 启发式搜索算法 — A*搜索小测试



1. 启发式搜索算法 — A*搜索



1. 启发式搜索算法 — A*搜索



- A*搜索**最优**的条件为：

$$0 \leq h(n) \leq h^*(n)$$

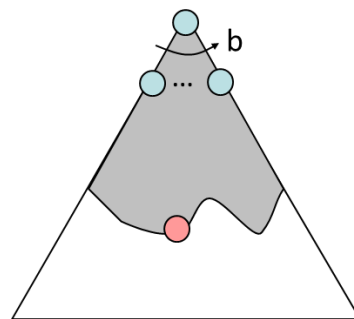
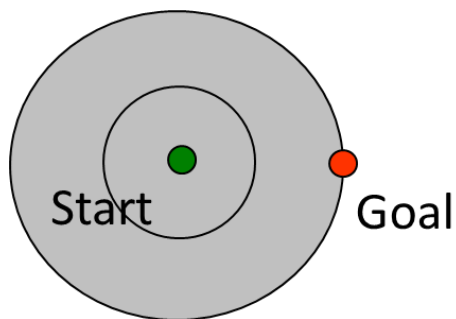
- 其中 $h^*(n)$ 是该状态到最终目标的**真正代价**



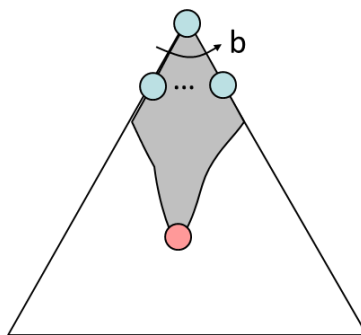
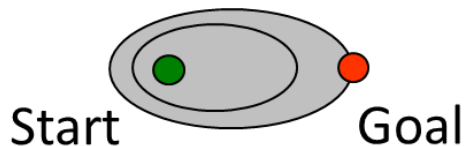
1. 启发式搜索算法 — A*搜索 vs. 一致代价搜索



- 一致代价搜索平等地在每个方向上搜索



- A*搜索主要向着目标节点进行扩展，但同时也会采取一些措施来确保其搜索结果的最优性



目 录

1

启发式搜索算法

2

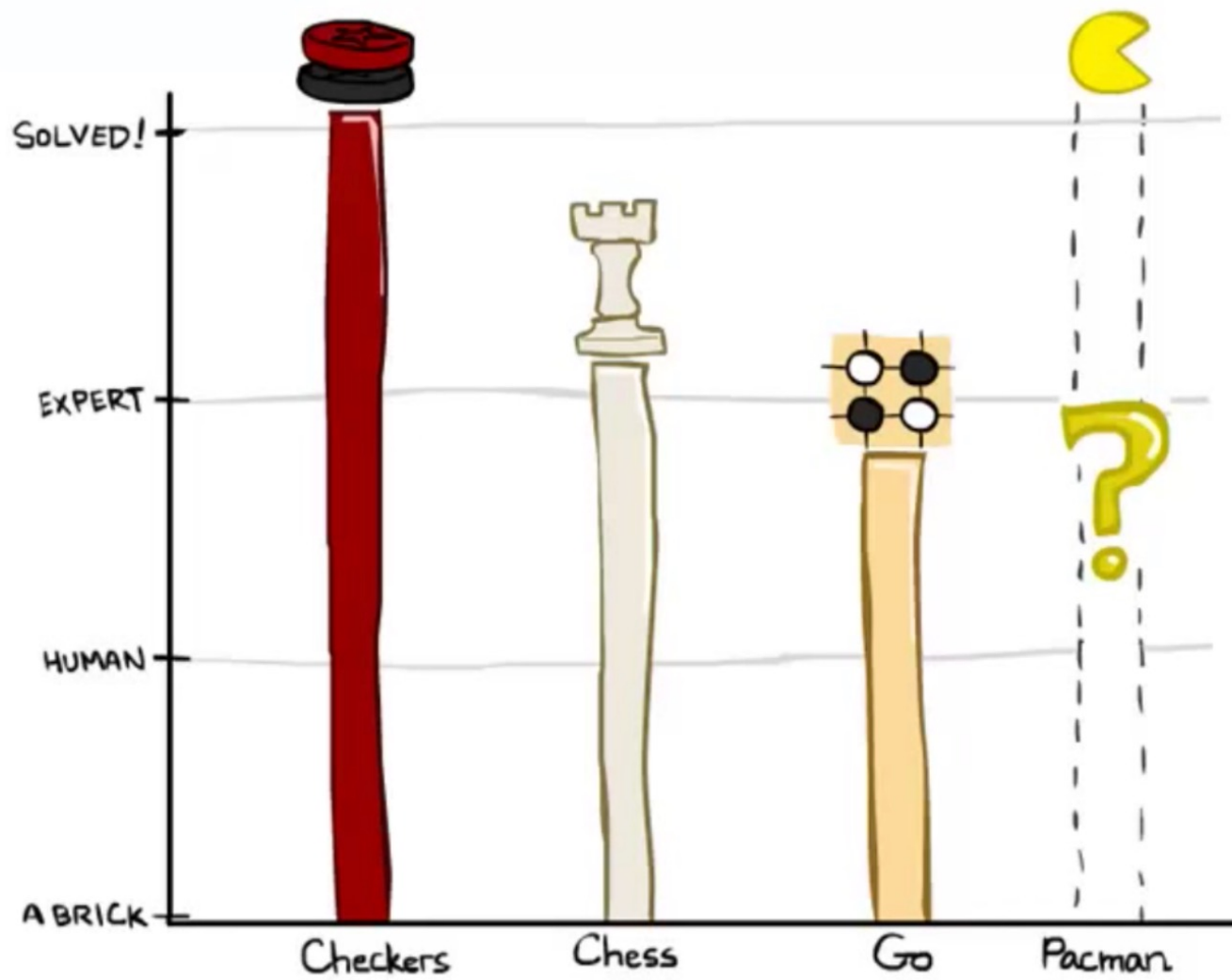
对抗搜索算法



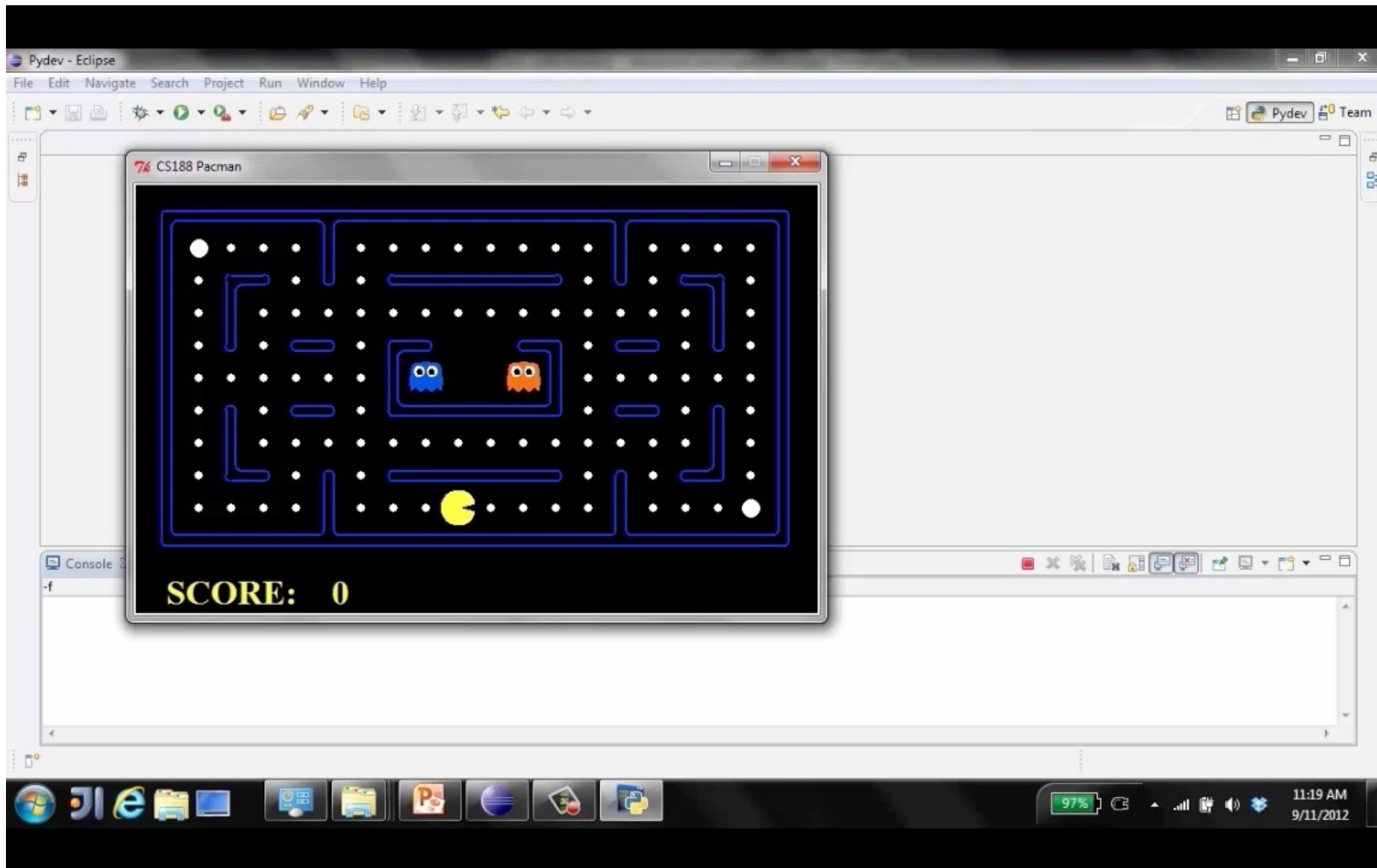
1. 对抗搜索简介



- 西洋跳棋
- 国际象棋
- 围棋
- 吃豆人



1. 对抗搜索简介 — 吃豆人游戏



1. 对抗搜索简介 — 博弈的分类



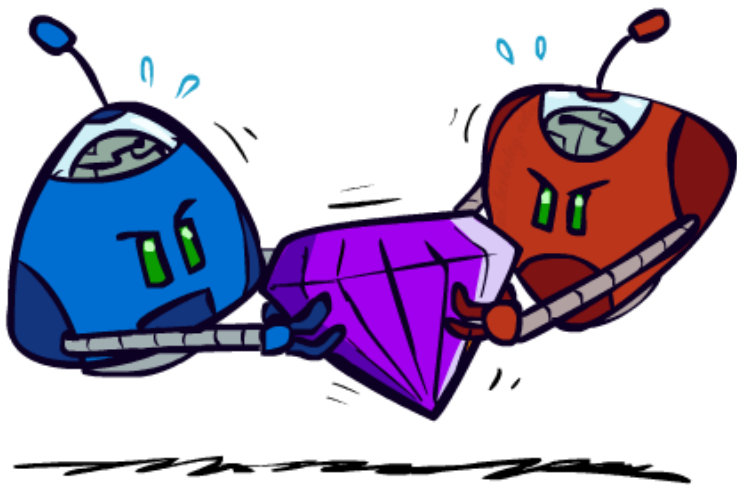
▪ 博弈的分类

- 确定性的还是随机的？
- 玩家的数量是一个、两个还是多个？
- 是否为零和博弈？
- 是否能得到完整的信息？



- 目标：设计算法，告诉玩家在每个**状态**应该采取什么**行动**

1. 对抗搜索简介 — 零和博弈



▪ 零和博弈

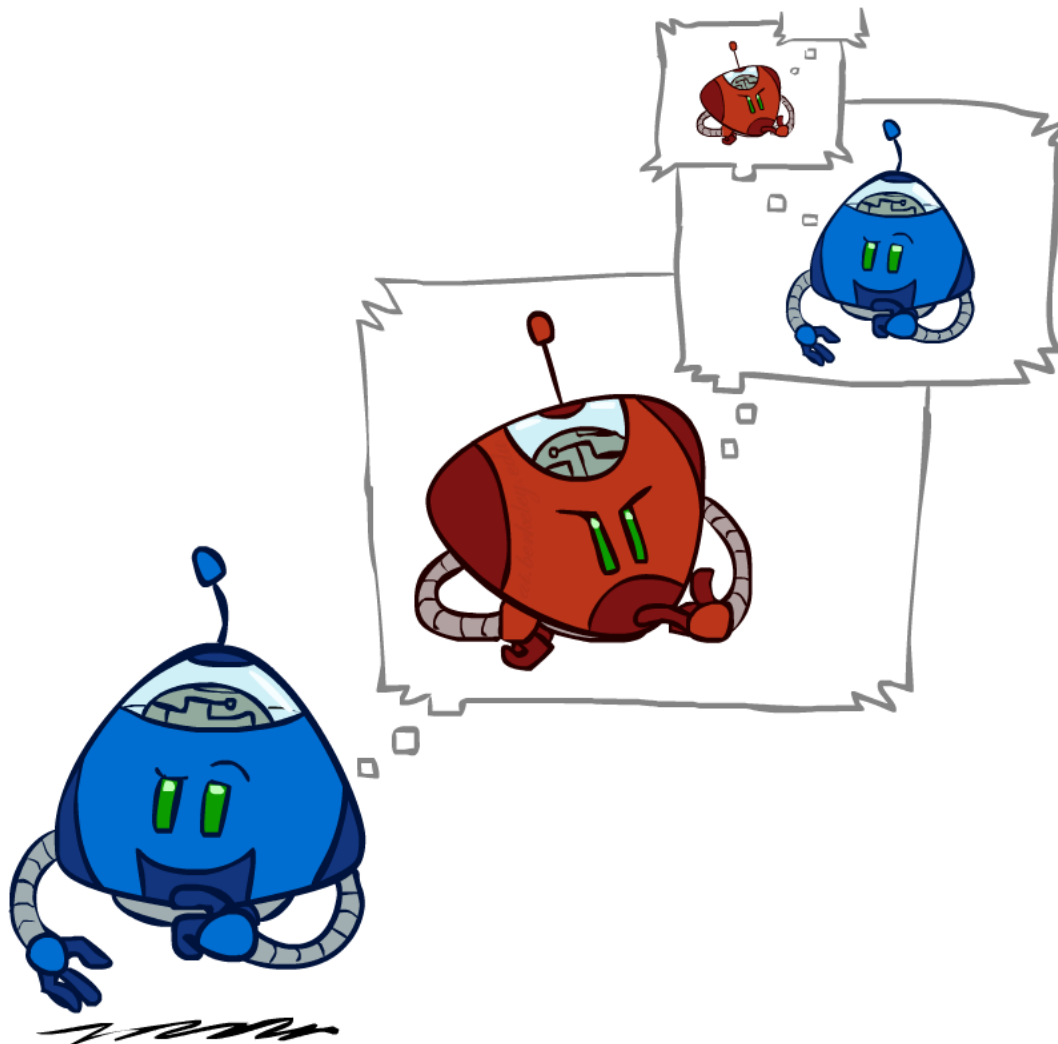
- 玩家有着对立的收益
- 一个玩家的收益增长代表着其他玩家收益减少
- 对抗的、纯竞争的



▪ 其他博弈

- 玩家有着独立的收益
- 玩家之间可以合作、互不干涉或者竞争，有更多的可能性
- 出现的晚于零和博弈

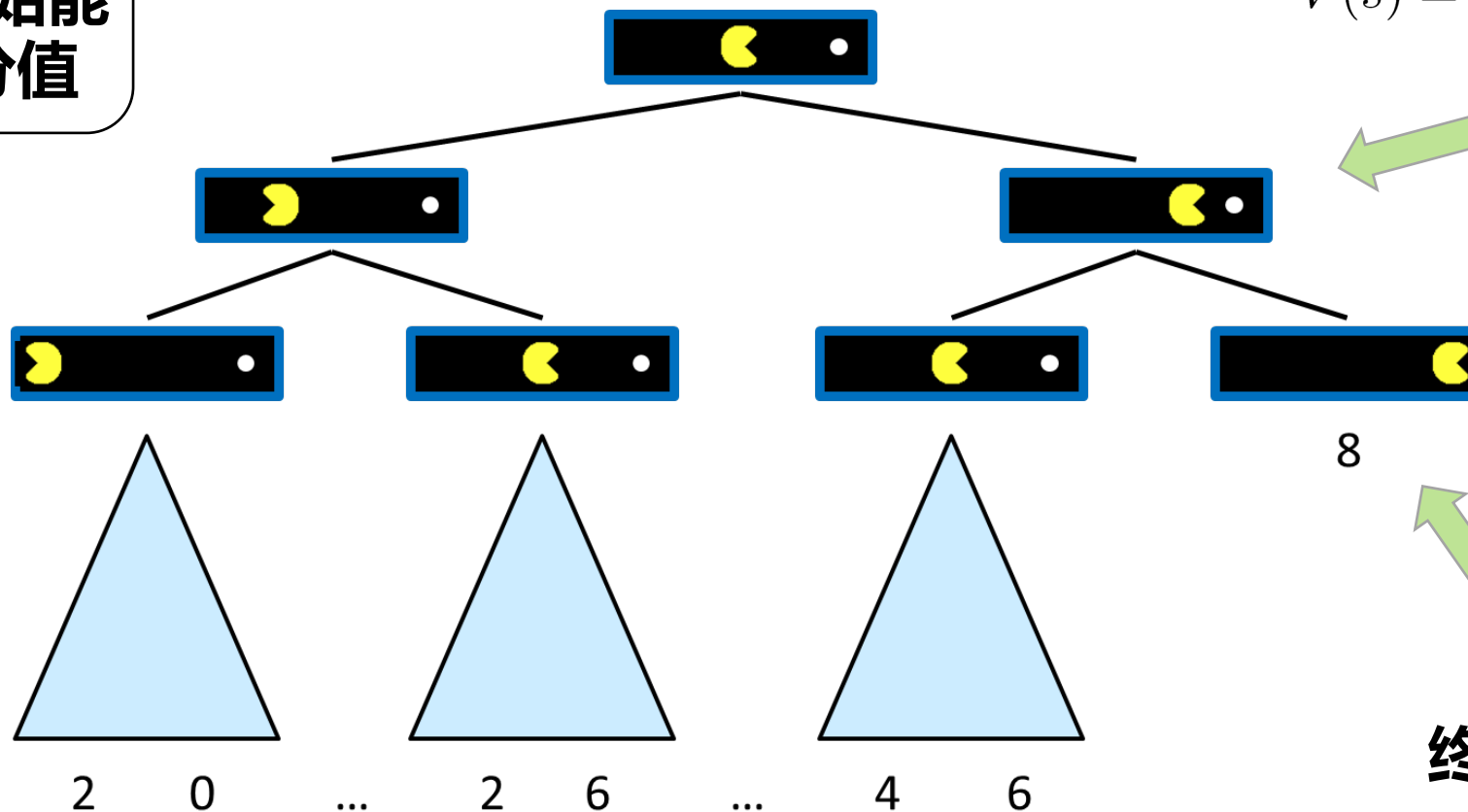
1. 对抗搜索简介 — 对抗搜索



1. 对抗搜索简介 — 单智能体搜索树



一个状态的分值是
从这个状态开始能
得到的最好分值



非终止状态：

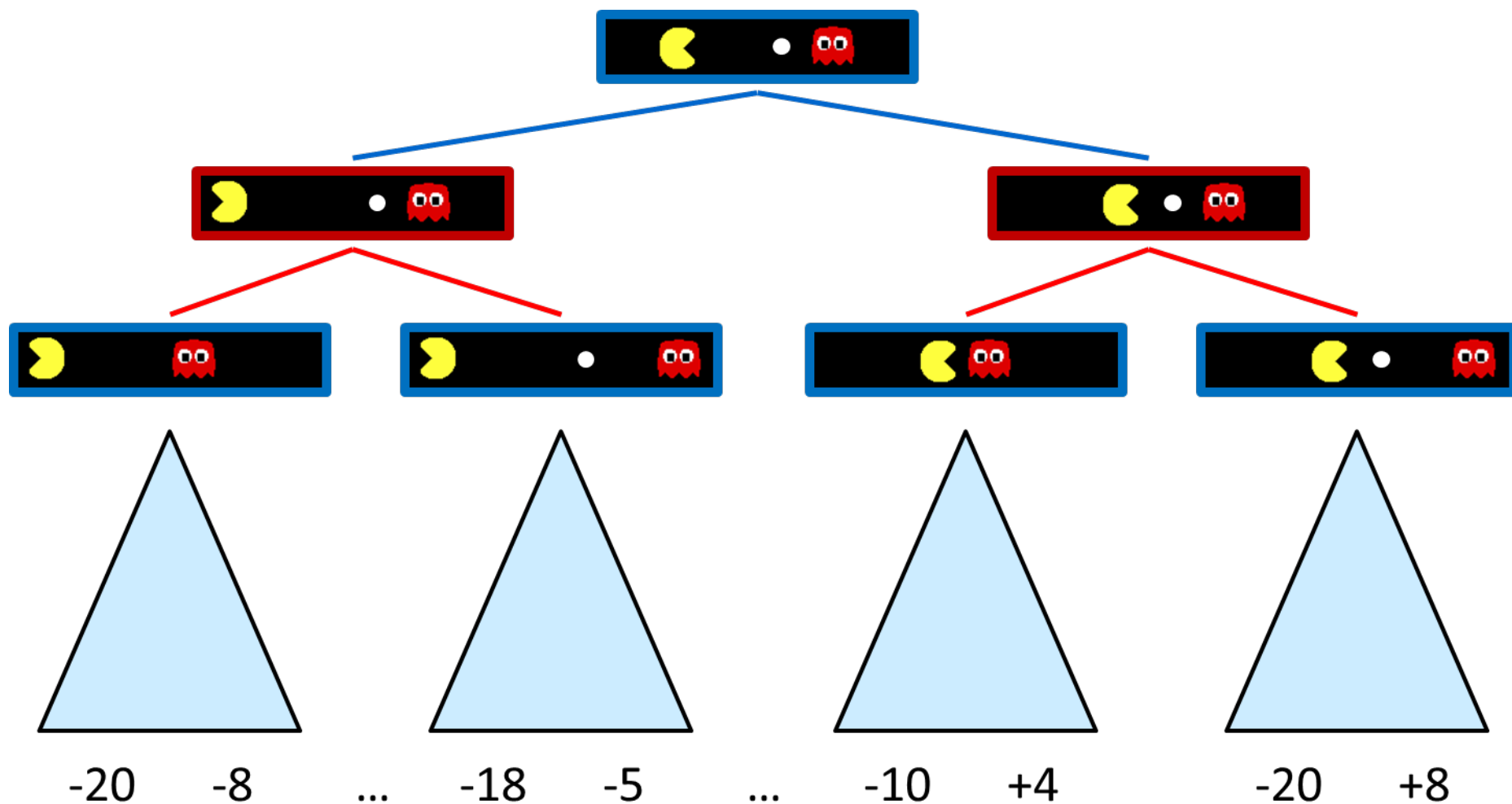
$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

终止状态：

$$V(s) = \text{known}$$



1. 对抗搜索简介 — 对抗博弈搜索树



2. 极小化极大算法 — 极小化极大值

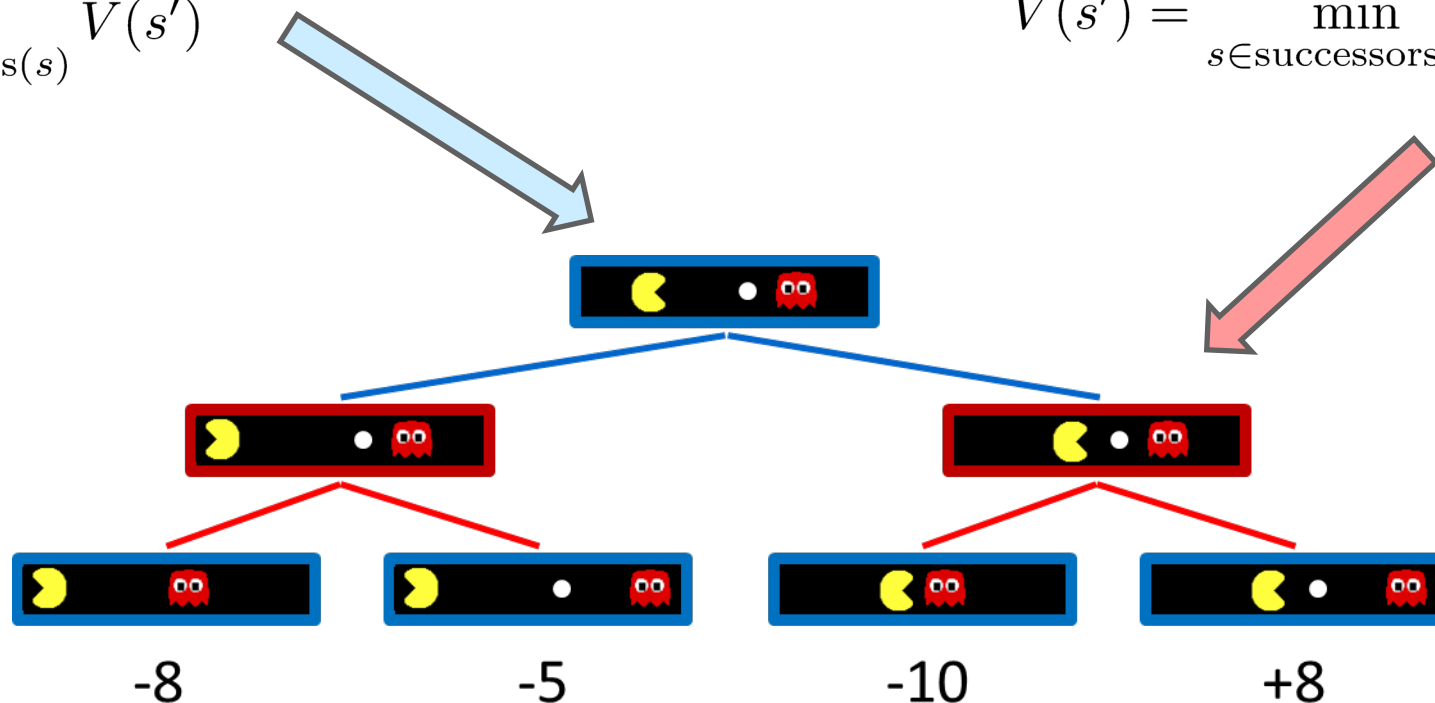


智能体的控制的状态：

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

对手控制的状态：

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

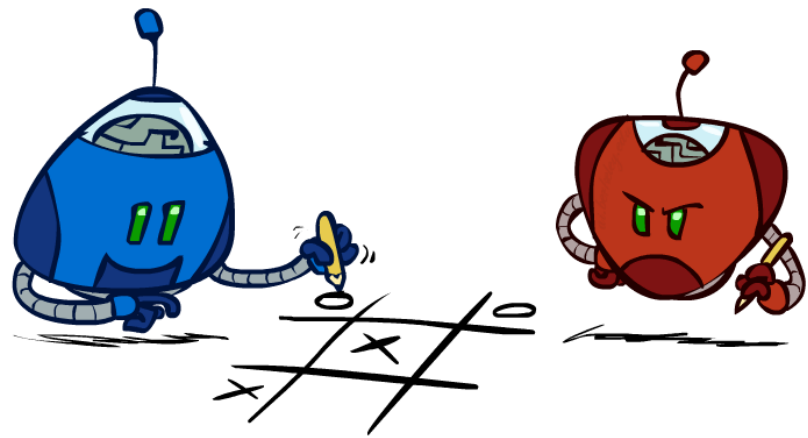
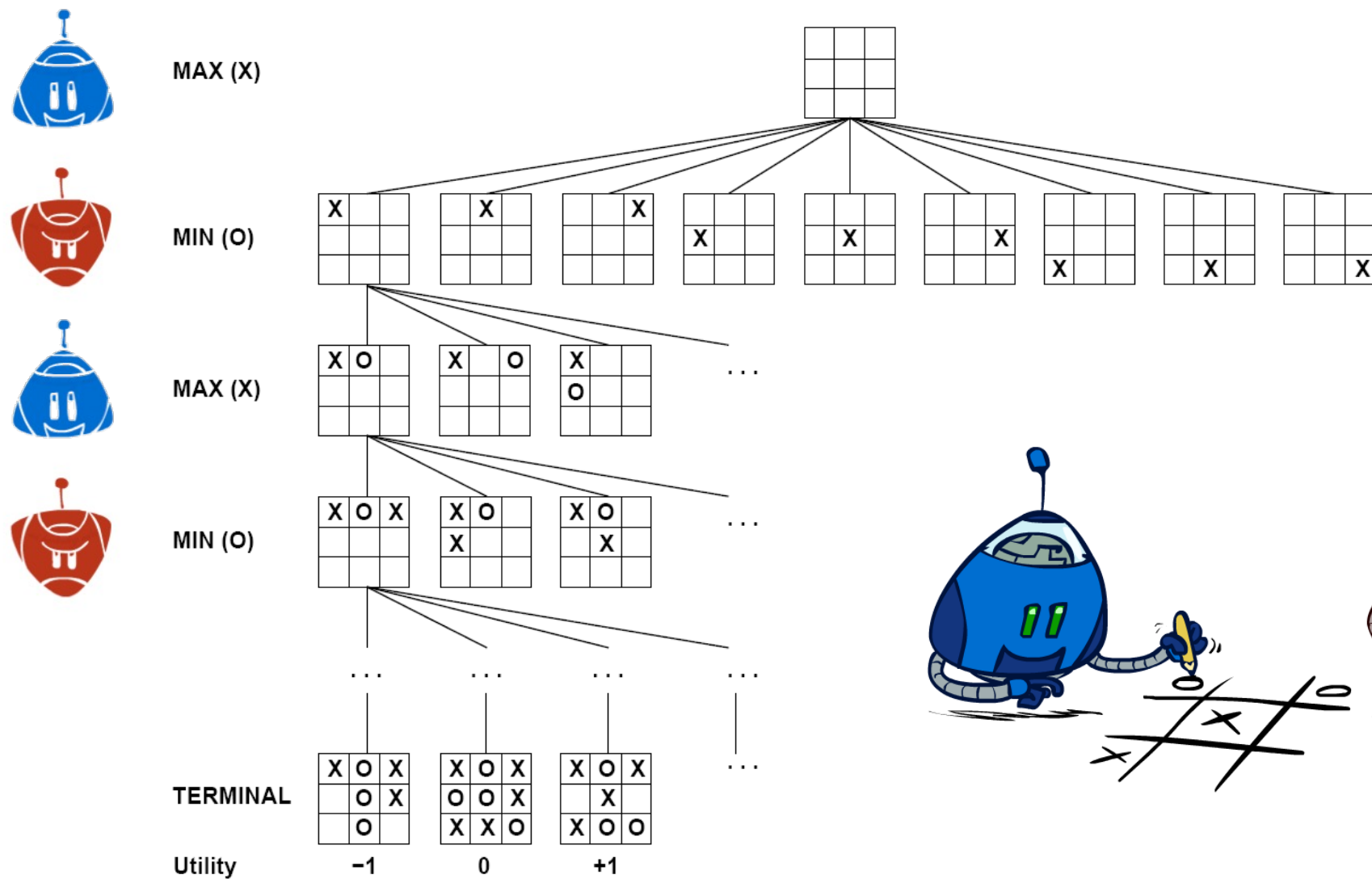


终止状态：

$$V(s) = \text{known}$$



2. 极小化极大算法 — 井字棋搜索树



2. 极小化极大算法 — Minimax对抗搜索



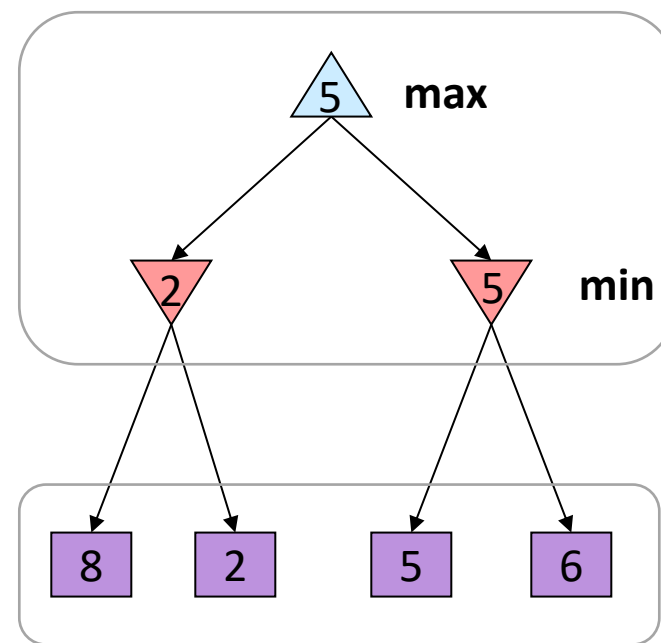
▪ 确定性的零和博弈

- 井字棋、围棋、象棋等
- 玩家A最大化自己的收益
- 玩家B最小化A的收益

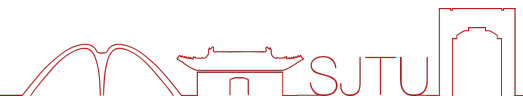
▪ 极小化极大（Minimax）搜索

- 一个状态空间搜索树
- 玩家依次做选择
- 每个节点的**极小化最大值**是指从这个状态开始，并**假设对手总能做出最优选择**的情况下能得到的最好分值

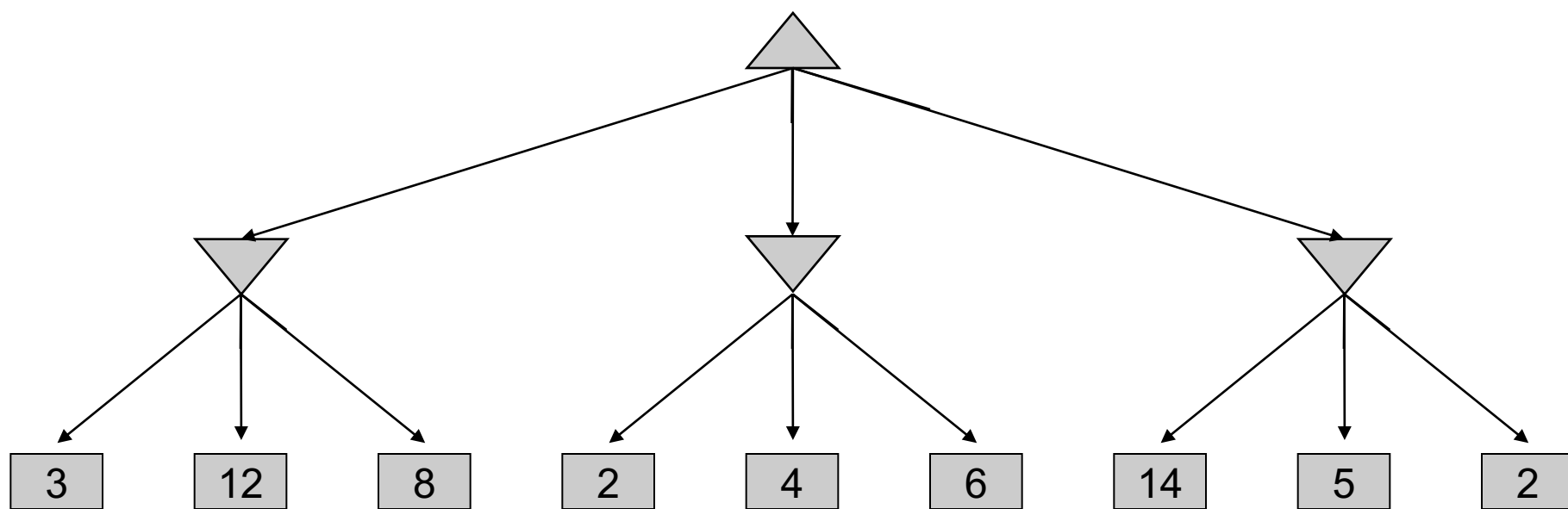
递归地计算**极大化极小值**



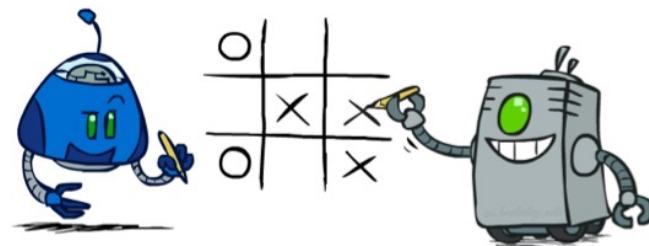
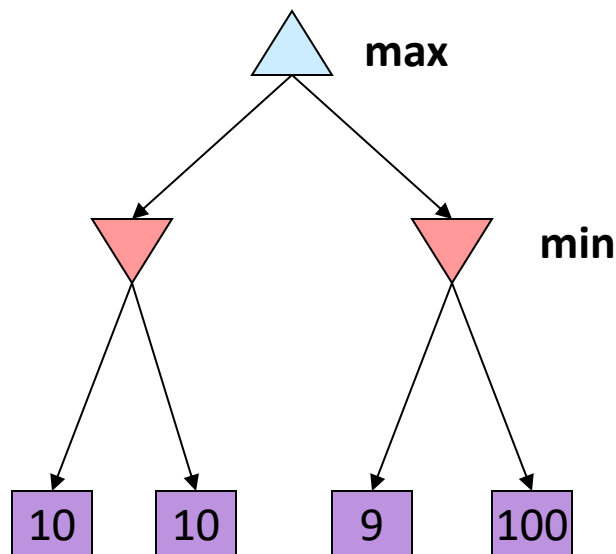
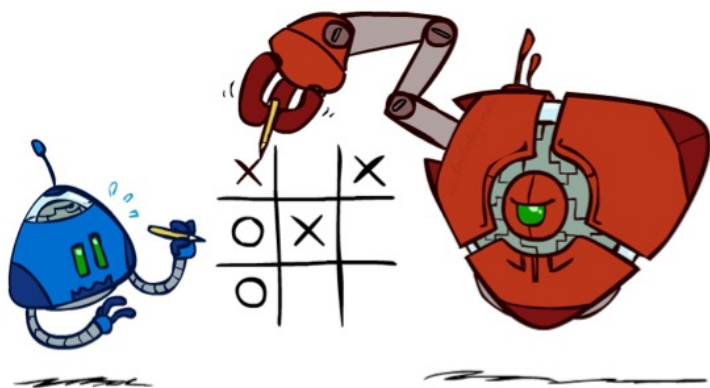
终止状态的分值



2. 极小化极大算法 — Minimax的例子

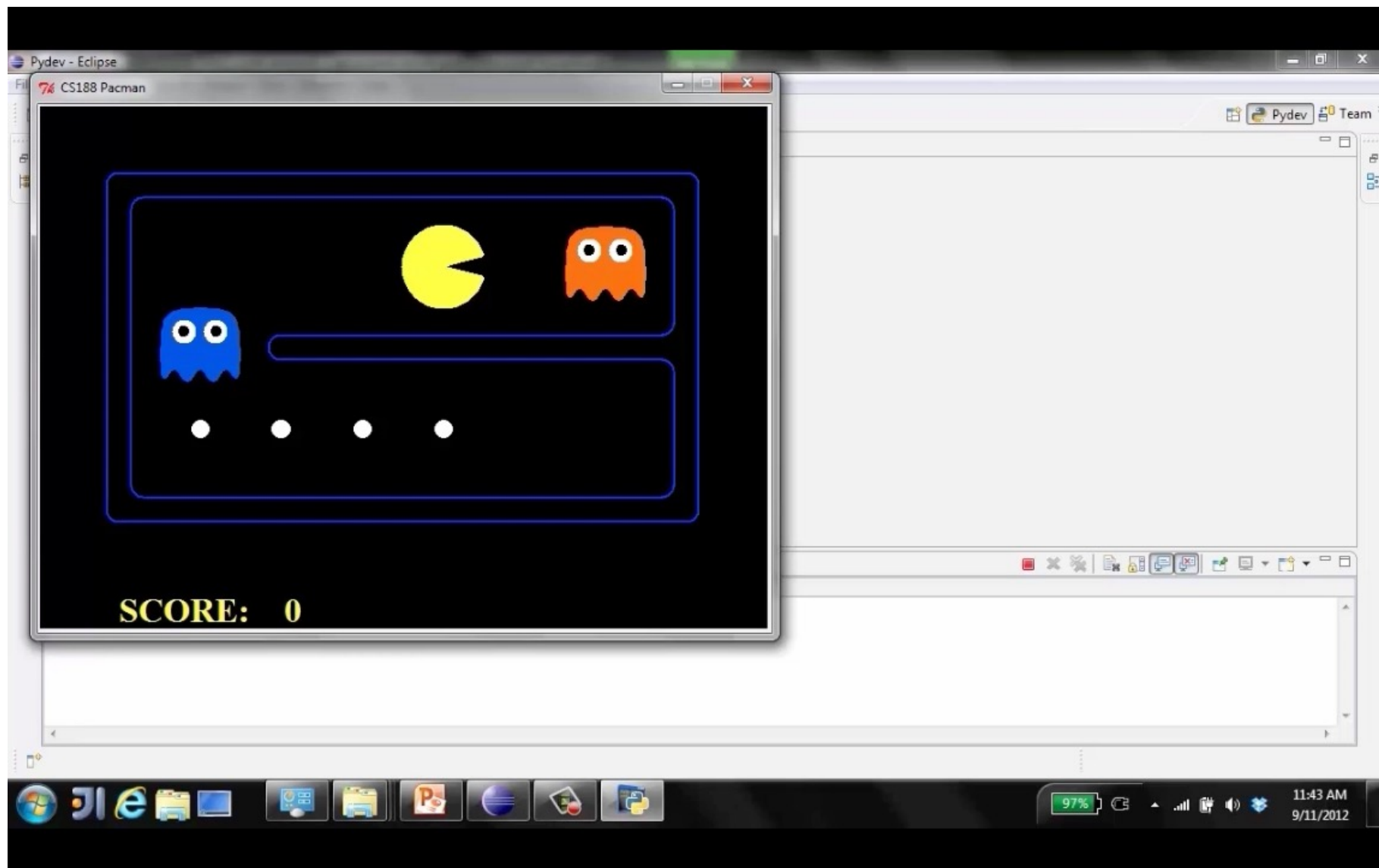


2. 极小化极大算法 — Minimax的特性

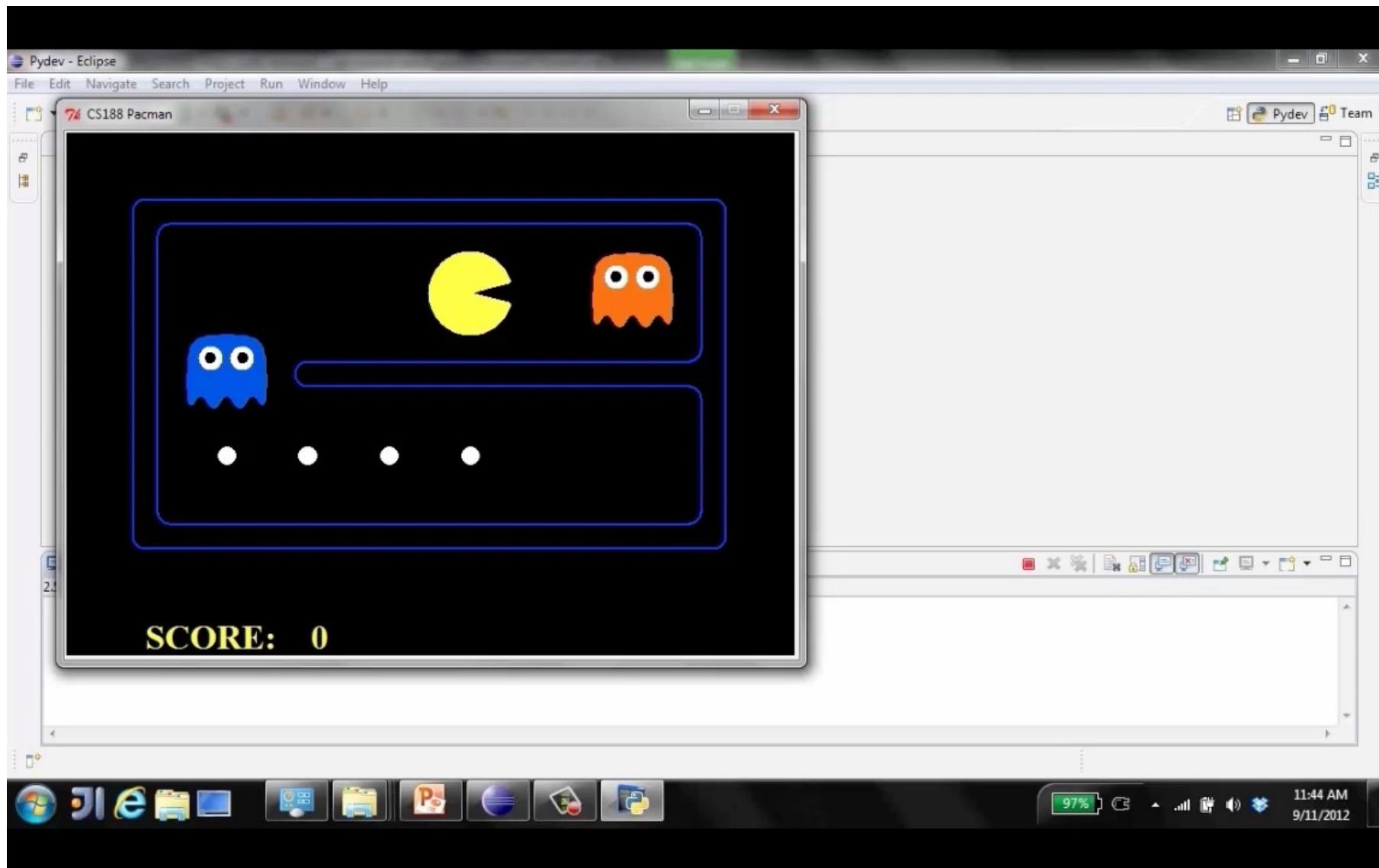


- Minimax搜索基于“对手总是会做出最优选择”这一假设

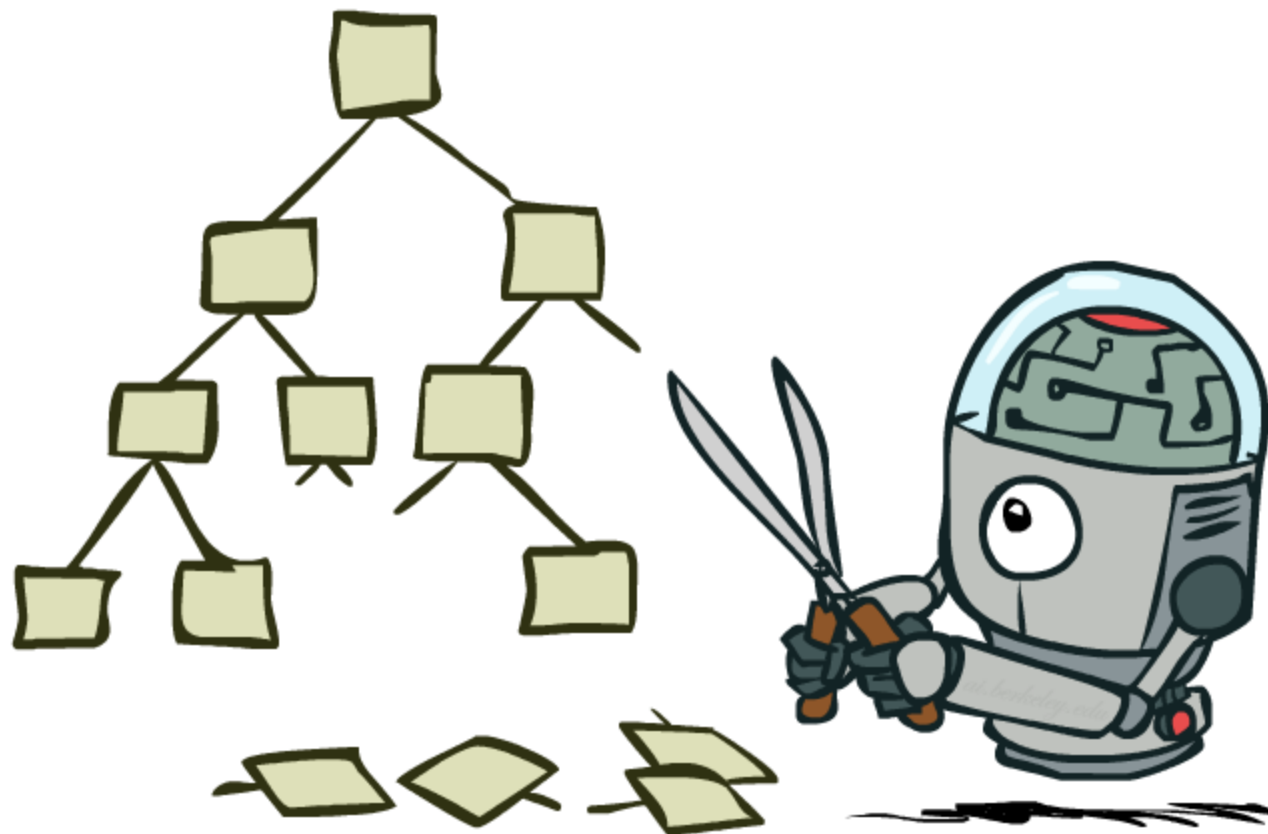
2. 极小化极大算法 — Minimax的特性



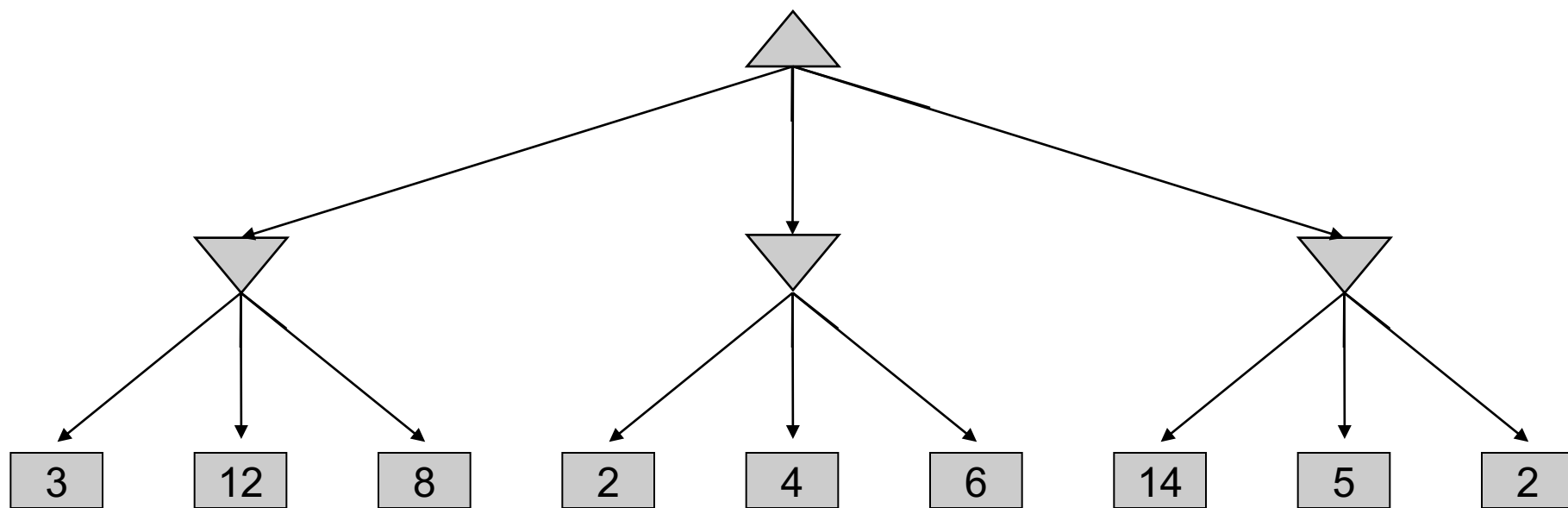
2. 极小化极大算法 — Minimax的特性



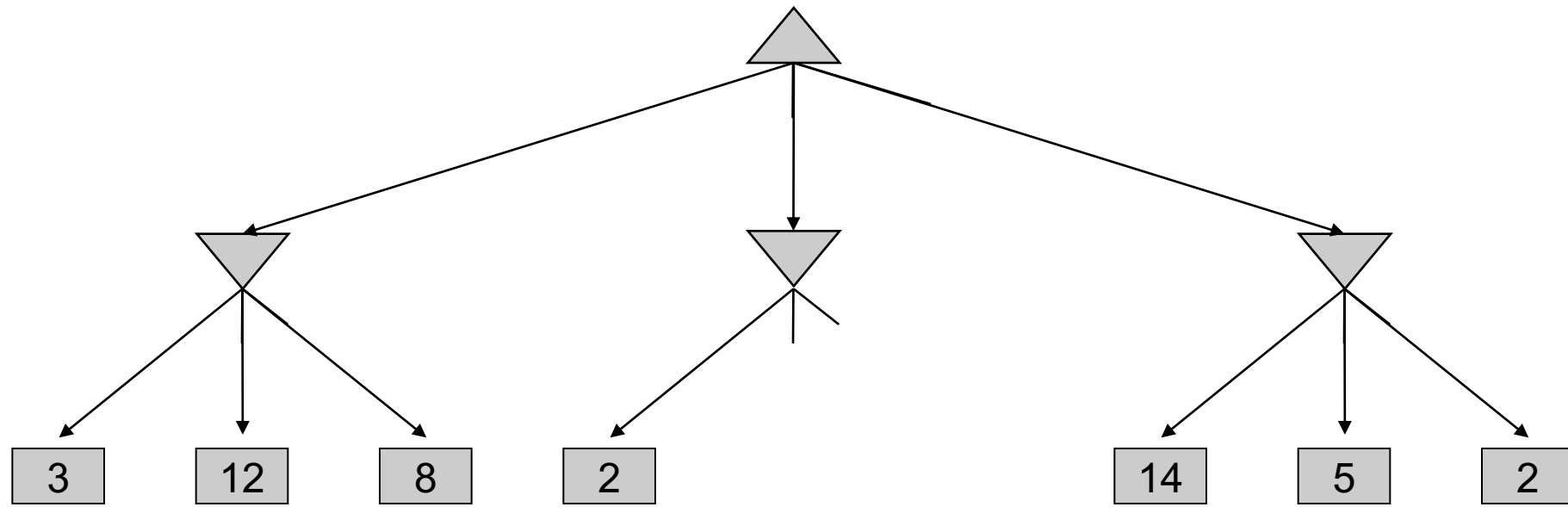
3. α - β 剪枝算法 — 博弈树剪枝



3. α - β 剪枝算法 — Minimax的例子



3. α - β 剪枝算法 — Minimax剪枝



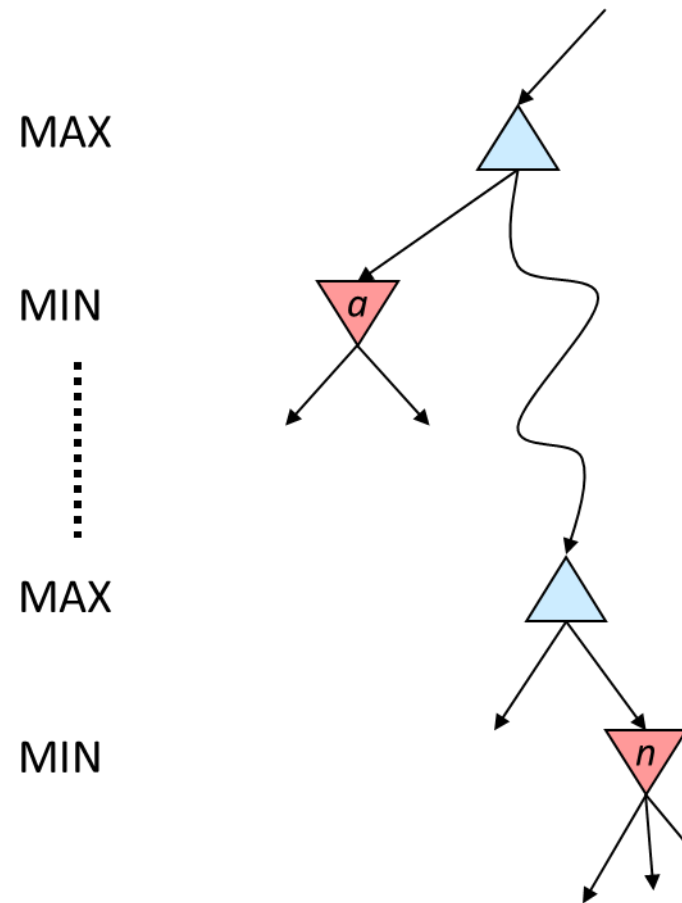
3. α - β 剪枝算法 — α - β 剪枝



▪ 剪枝原理（MIN节点）

- 假设我们正在计算节点n的min-value
- 我们需要遍历n的子节点
- 假设 α 是n上方某个MAX节点目前得到的最好的结果
- 若n当前的取值小于等于 α ，则上方的MAX将不会取到n的值，我们便可以不用再遍历n剩下的节点

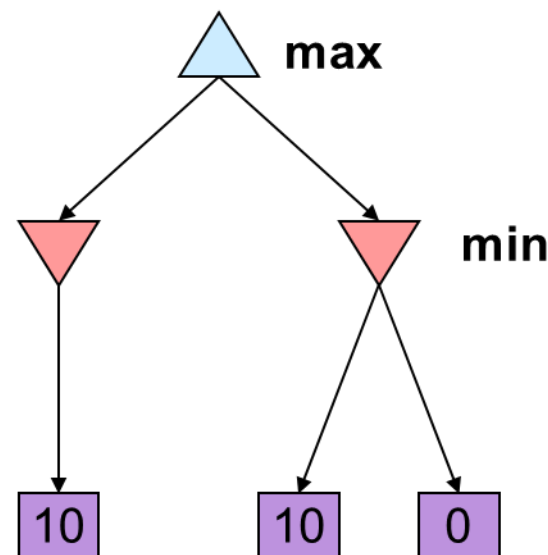
▪ 对于MAX节点的剪枝同理可得



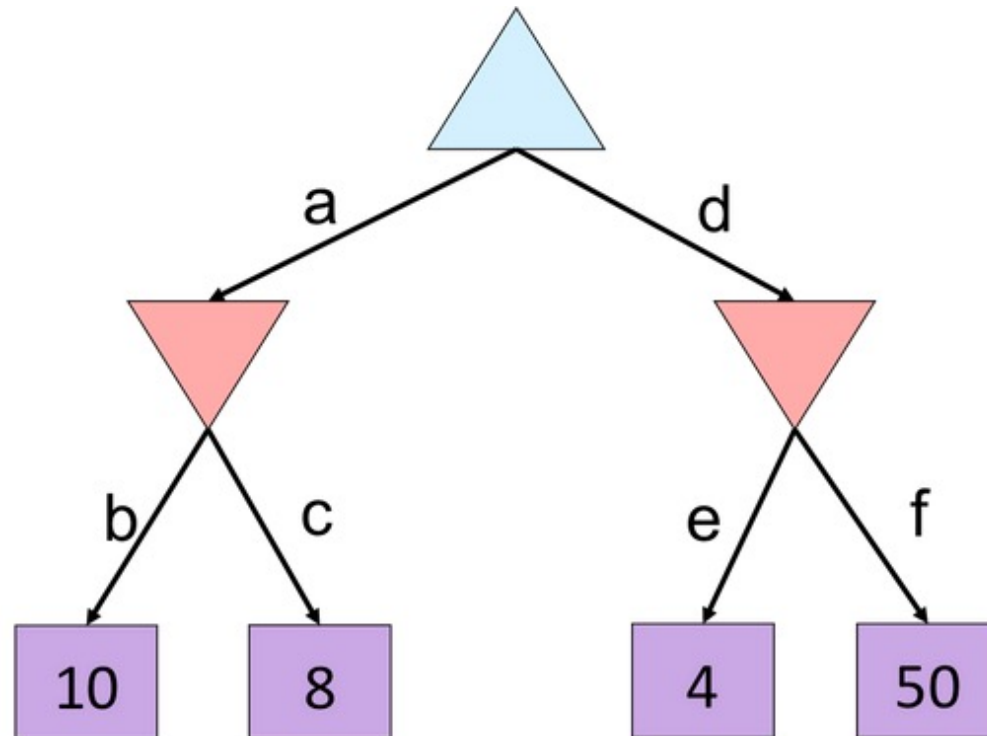
3. α - β 剪枝算法 — α - β 剪枝的特性



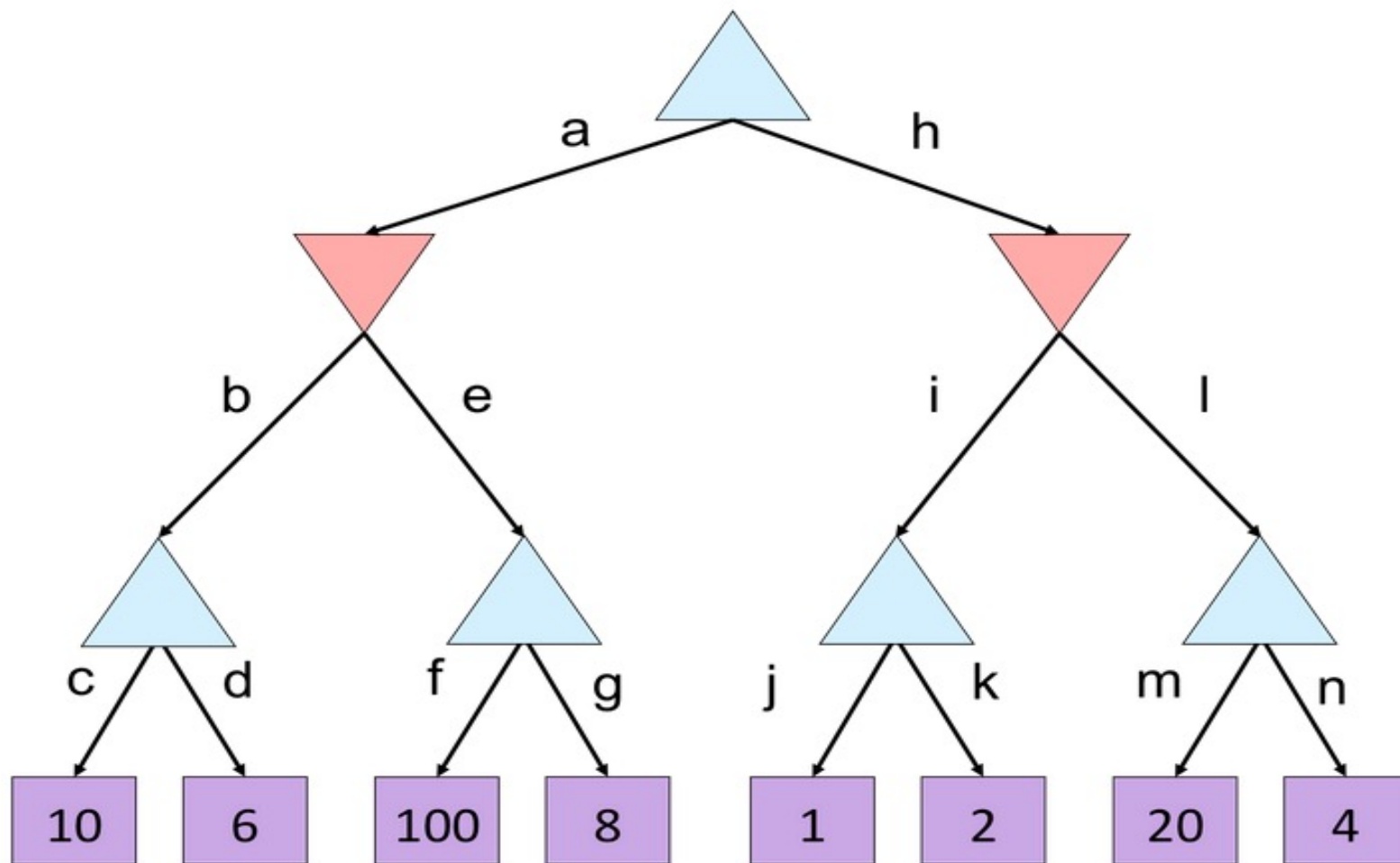
- α - β 剪枝**不会影响**根节点的极小化极大值
- 非根节点的值可能是错误的
- 最好的情况下
 - 搜索深度可以**翻倍**
 - 但是仍然无法完整地搜索围棋等复杂的游戏



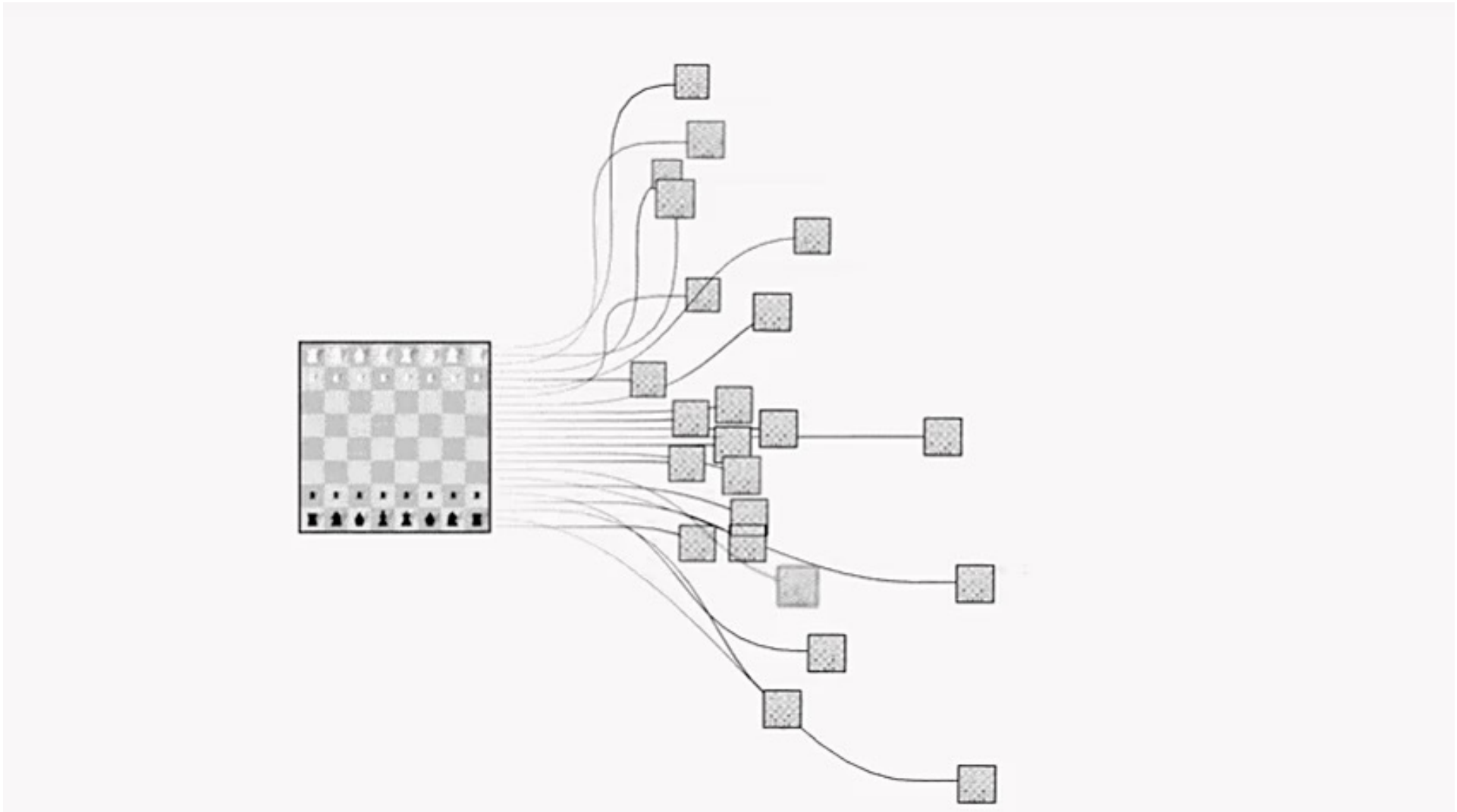
3. α - β 剪枝算法 — 小测试

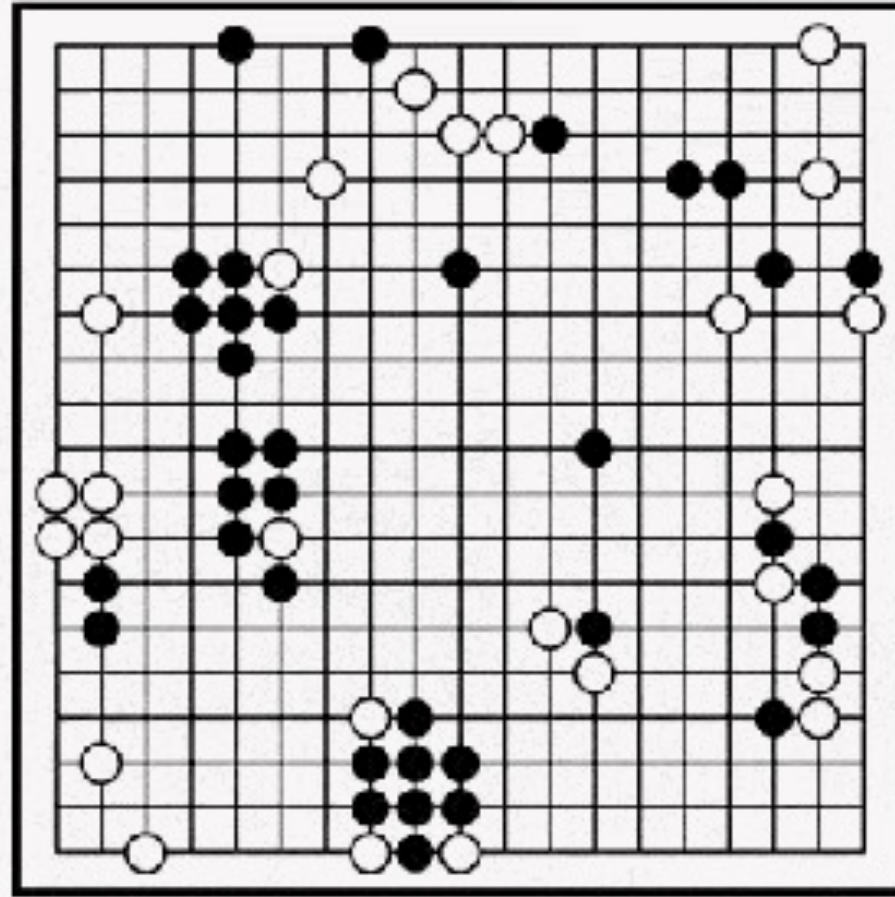


3. α - β 剪枝算法 — 小测试



例子：国际象棋







上海交通大学

SHANGHAI JIAO TONG UNIVERSITY



人工智能研究院

Artificial Intelligence Institute

谢谢！

饮水思源 爱国荣校